00. A4_1 평가 Q&A

NumPy TF-IDF 검색 · 분류
평가표 16 문항과 보너스 3 항목

A4_1 · NumPy TF-IDF 검색과 분류
2026-11-12 문서 및 실측 기준

1-1A. TF-IDF 참조 검증 조건

전처리와 열 순서가 같아야 수치 오차를 비교할 수 있다

비교 조건	직접 구현과 sklearn 의 공통 기준
전처리	동일한 TextPreprocessor.tokenize 사용
어휘	Train 에서 학습 , 알파벳순으로 열 고정
TF / IDF	Raw TF / smooth_idf=True
정규화 / 자료형	norm='l2' / dtype=np.float64
추가 설정	sublinear_tf=False, 영벡터 보존

토큰 · 어휘 ·TF·DF·IDF 를 확인한 뒤 최종 벡터를 대조한다

근거 : 평가 Q&A 및 저장소 실측 산출물 · 상세 출처는 마지막 장

1-1B. 허용 오차와 원문 실측

비교 대상	최대 절대 오차	판정 기준
Train 3,029 건	2.3315e−15	1e−6 이내
Test 758 건	8.3267e−16	1e−6 이내
IDF	0	동일
검증 질의	0	동일

1e−15 수준은 측정값의 규모이며 허용 기준은 1e−6 이다

근거 : 평가 Q&A 및 저장소 실측 산출물 · 상세 출처는 마지막 장

1-2. NumPy 코사인 검색

$$s(d,q) = (d \cdot q) / (\|d\|_2 \|q\|_2) = \hat{d} \cdot q_norm$$
$$scores = X_train @ q$$

공통 공간	문서와 질의에 같은 Train 어휘 ·IDF 를 적용하고 L2 정규화한다 .
Top-K 정책	양수 점수만 선택한다 . 점수 내림차순 , 동점은 문서 ID 오름차순이다 .
경계 입력	빈 입력 , 불용어만 있는 입력 , OOV 질의는 빈 리스트를 반환한다 .
우주 질의 실측	Top-5 ID 는 1934, 2609, 755, 2145, 1725 이며 첫 점수는 0.4737 이다 .

상위 문서의 관련성을 관찰했으며 Precision@5 정답셋 평가는 아직 수행하지 않았다

근거 : 평가 Q&A 및 저장소 실측 산출물 · 상세 출처는 마지막 장

1-3A. 분류 성능과 고정 분할

층화 8:2 분할 , random_state=42, 동일한 TF-IDF 표현

지표 또는 설정	원문 실측과 설정
Test 정답 / 전체	701 / 758
Accuracy	92.4802%
Macro-F1	0.925329
전체 오분류	57 건
분류기	LogisticRegression(max_iter=1000, random_state=42)

정확도와 클래스별 지표 , 혼동 행렬을 함께 읽는다

근거 : 평가 Q&A 및 저장소 실측 산출물 · 상세 출처는 마지막 장

1-3B. Accuracy · F1 · 혼동 행렬 해석

예측 클래스

실제 클래스	그래픽	야구	우주	중동 정치
그래픽	**179**	4	8	0
야구	4	**177**	10	1
우주	5	5	**178**	3
중동 정치	1	8	8	**167**

92.4802%

701 / 758 건 정답
Macro-F1 0.925329

우주로 잘못 예측한 문서
8 + 10 + 8 = 26 건

세로는 정답 , 가로는 예측이다 . 대각선 밖의 57 건이 오분류다 .

클래스 순서는 target_names 와 같으며 전체 57 건의 오류 중 다섯 사례를 분석했다

근거 : 평가 Q&A 및 저장소 실측 산출물 · 상세 출처는 마지막 장

1-3C. Precision · Recall · F1 의 계산

혼동 행렬을 한 클래스 대 나머지의 이진 문제로 읽는다

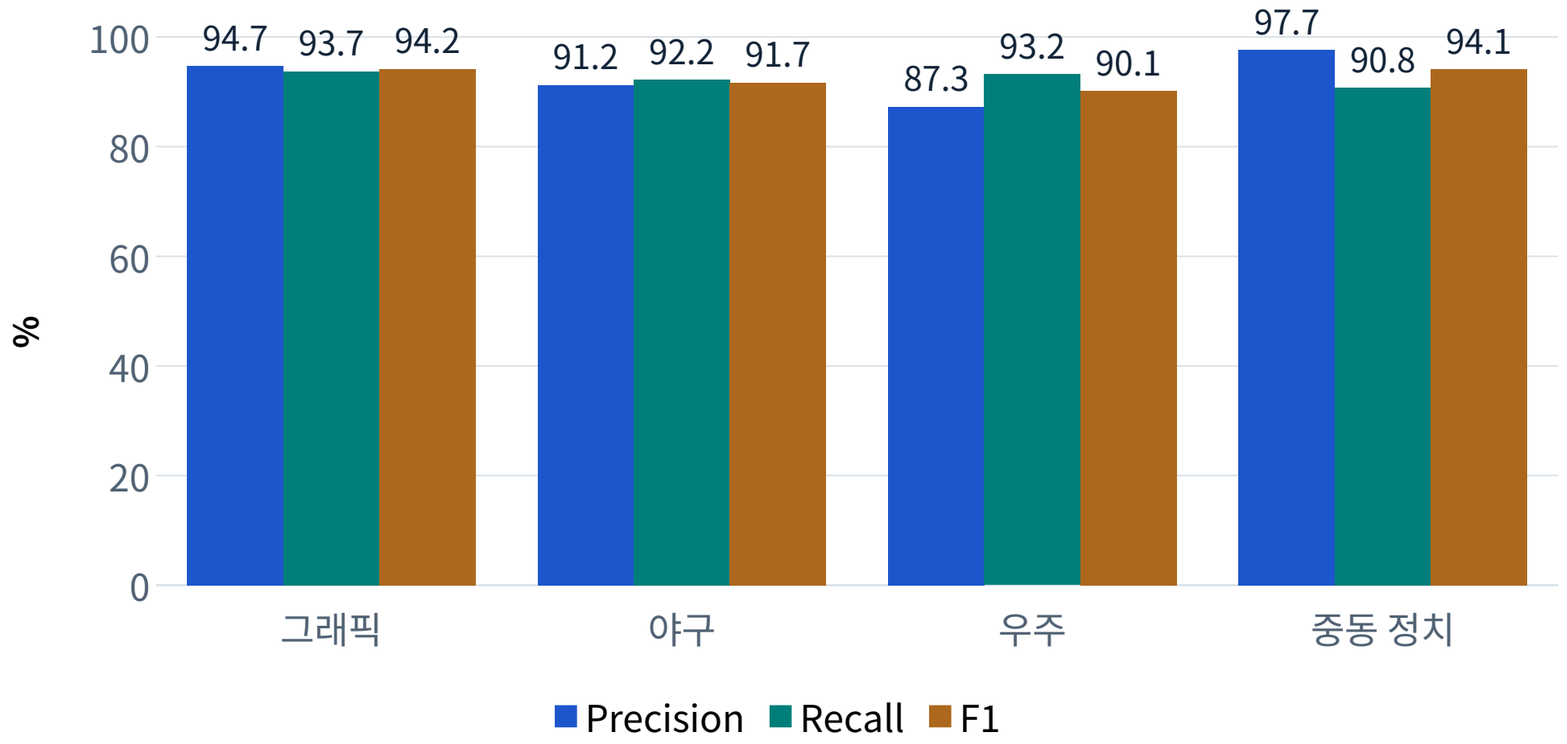
Precision = TP / (TP + FP)
Recall = TP / (TP + FN)
F1 = 2 × Precision × Recall / (Precision + Recall)

우주 클래스로 읽기	TP = 178, FP = 26, FN = 13. 우주 예측 204 건 중 178 건이 정답이고 , 실제 우주 191 건 중 178 건을 맞혔다 .
Macro-F1	각 클래스별로 F1 을 계산한 뒤 네 값을 같은 비중으로 평균한다 . 이 실측에서는 0.925329 이다 .
Accuracy	대각선의 정답 수를 전체 Test 수로 나눈다 . (179 + 177 + 178 + 167) / 758 = 92.4802%.

근거 : 평가 Q&A 및 저장소 실측 산출물 · 상세 출처는 마지막 장

1-3D. 클래스별 분류 성능

전체 정확도와 별도로 어느 클래스에서 오류가 늘어나는지 본다



우주 클래스

Precision 87.25%
Recall 93.19%
F1 90.13%

우주 문서를 많이 찾지만
다른 주제도 우주로
예측한 사례가 있다 .

Macro-F1 은 네 클래스 F1 의 산술평균이다 . 표본 수로 가중한 평균과 구분한다 .

근거 : 평가 Q&A 및 저장소 실측 산출물 · 상세 출처는 마지막 장

1-4A. 실제 토큰화 규칙

```
from src.preprocess import TextPreprocessor

p = TextPreprocessor(stopwords=set())
p.tokenize("Space, SATELLITE! a 3D_x")
# ["space", "satellite", "3d_x"]
```

소문자화와 추출

r'(?u)\b\w\w+\b' 로 두 글자 이상의 Unicode 단어를 추출한다 .

문자 범위

숫자와 밑줄도 \w 에 속한다 . 특수문자를 모두 없애는 규칙으로 설명하면 부정확하다 .

동일한 검증 기준

NumPy 와 sklearn 모두 같은 analyzer 를 사용한다 . 불용어 123 개를 a4-1-en-v1 로 고정했다 .

전처리 규칙과 불용어 해시를 고정해 실험 조건을 재현한다

근거 : 평가 Q&A 및 저장소 실측 산출물 · 상세 출처는 마지막 장

1-4B. 전처리에서 사라지는 정보

처리 대상	현재 규칙의 영향	후속 비교
한 글자 토큰	두 글자 미만을 제외	도메인 토큰 보존 여부
C++ 같은 표기	기호 · 짧은 토큰 정보 손실 가능	기술 용어 전용 규칙
no / nor / not	불용어 제거로 부정 의미 손실	부정어를 보존한 실험
한국어 어미 · 조사	영어 기준으로 의미 단위 분리 부족	형태소 분석과 공백 토큰 비교

규칙 변경의 개선 여부는 같은 분할에서 어휘 ·OOV· 성능을 다시 측정해야 한다

근거 : 평가 Q&A 및 저장소 실측 산출물 · 상세 출처는 마지막 장

1-5A. 실제 Test 오분류 다섯 건

Test ID	실제 / 예측	점수 차이 Δ	분석 핵심
3	중동 정치 / 우주	+0.0123	정치 단서보다 범용 어휘와 절편의 합이 큼
5	그래픽 / 우주	+0.4278	base·system, 짧은 대화 맥락과 토큰 손실
13	그래픽 / 야구	+0.2199	잔존 서명과 일반 표현의 기여
19	야구 / 우주	+0.0233	약어 · 기사 날짜에 필요한 이전 대화 맥락
25	중동 정치 / 우주	+0.1136	한 토큰만 남아 절편의 영향이 큼

Test 에서 실제로 틀린 문서를 ID 순으로 추출했다 . 원인 해석과 개선 성능 검증은 별개다

근거 : 평가 Q&A 및 저장소 실측 산출물 · 상세 출처는 마지막 장

1-5B. 단어 기여도와 절편 분해

$$\Delta = (b_{\hat{y}} - b_y) + \Sigma_t\, x_t (w_{\hat{y},t} - w_{y,t})$$
$$\Delta_{25} = +0.2461 - 0.1325 = +0.1136$$

점수의 의미 Δ 는 예측 클래스와 실제 클래스의 선형 점수 차이다 . 양수 기여는 예측 클래스를 지지한다 .

ID 25 사례 전처리 후 남은 한 토큰은 실제 클래스를 지지하지만 , 더 큰 절편 차이가 최종 판정을 바꾼다 .

자료 공개 범위 공개 JSON 은 문서 ID·feature ID· 수치를 사용한다 . 전체 인명 · 서명 수동 검수는 남은 과제다 .

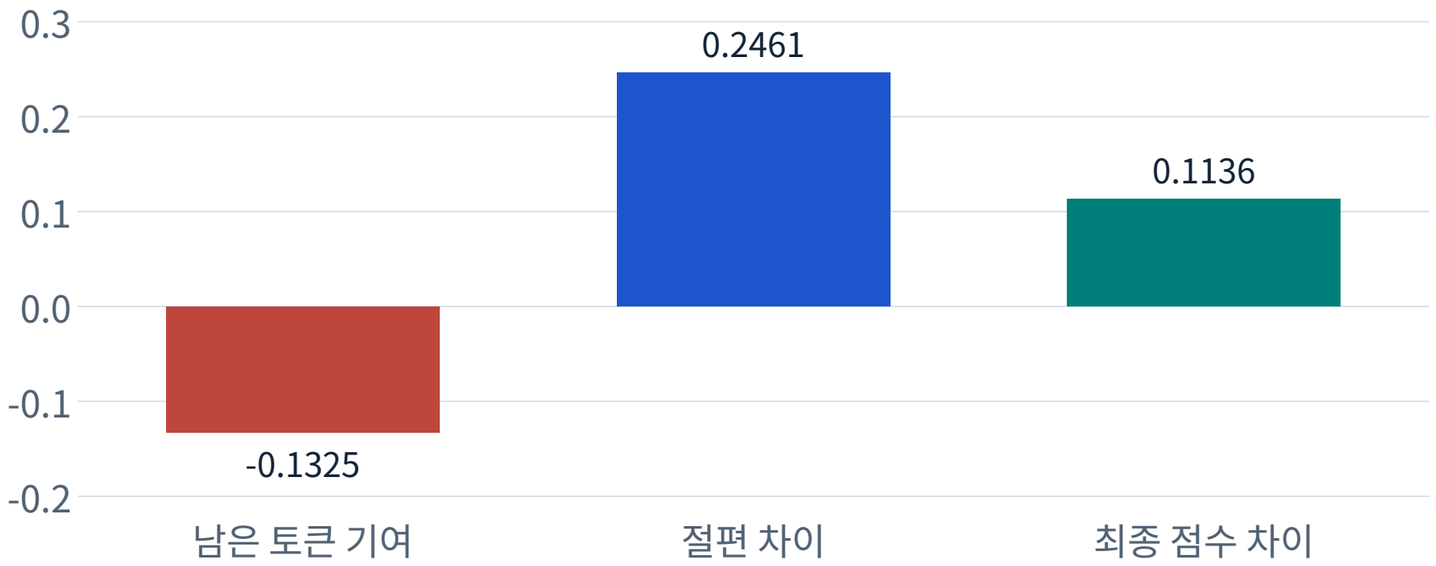
관찰한 모델 기여도를 근거로 원인을 해석하며 개선 효과는 별도 실험으로 확인한다

근거 : 평가 Q&A 및 저장소 실측 산출물 · 상세 출처는 마지막 장

1-5C. Test ID 25 의 기여도 분해

한 토큰의 기여와 절편 차이를 더하면 최종 점수 차이가 된다

$$s_pred - s_true = \Sigma_j x_j (w_pred,j - w_true,j) + (b_pred - b_true)$$



Test ID 25

정답 : 중동 정치
예측 : 우주

전처리 후 한 토큰만 남았다 .
토큰의 정답 지지보다
절편 차이가 더 컸다 .

근거 : 평가 Q&A 및 저장소 실측 산출물 · 상세 출처는 마지막 장

1-6. CLI· 함수 호출과 재현성

```
python main.py --build-index
python main.py --query "space satellite launch orbit" --topk 5

from main import search
results = search("space satellite launch orbit", top_k=5)
```

반환 계약

(score, doc_id, text_snippet) 목록을 반환한다 . 스니펫은 150 자 이하이다 .

저장과 재로딩

CLI 는 artifacts/index.pkl, 노트북은 NPY· JSON 인덱스를 사용한다 .

원문 실행 근거

요청된 네 CLI 명령 종료 코드 0. 회귀 검사 59 개 , 노트북 28 셀 · 코드 14 셀 실행을 확인했다 .

프로젝트 루트에서 같은 가상환경 또는 노트북 커널을 사용한다

근거 : 평가 Q&A 및 저장소 실측 산출물 · 상세 출처는 마지막 장

2-1. 모듈별 책임과 입력 · 출력

모듈	담당 책임	분리의 효과
dataset.py	문서 · 레이블 · 클래스명 로딩	다른 데이터셋의 입력 형식 수용
preprocess.py	토큰화와 불용어 처리	언어 · 도메인 규칙 교체
tfidf.py	어휘 ·IDF 학습과 벡터화	검색 · 분류가 같은 표현 재사용
search_engine.py	점수 계산과 Top-K 정렬	동점 · 빈 입력 정책 독립 검증
classifier.py	학습 · 지표 · 오분류 분석	표현 변경과 분류 오류를 분리

입력 · 출력 계약을 유지하면 문제 발생 단계를 좁히고 한 구성 요소만 바꾸기 쉽다

근거 : 평가 Q&A 및 저장소 실측 산출물 · 상세 출처는 마지막 장

2-2A. TF·DF·IDF· 정규화 계산

```
import numpy as np

tf = np.array([[2, 1], [0, 1]], dtype=np.float64)
df = np.count_nonzero(tf, axis=0)
idf = np.log((1 + len(tf)) / (1 + df)) + 1
weighted = tf * idf
norms = np.linalg.norm(weighted, axis=1, keepdims=True)
x = np.divide(weighted, norms,
              out=np.zeros_like(weighted), where=norms != 0)
```

열별 계산

DF=[1, 2], IDF=[1.405465, 1.0]. TF 의 각 열에 해당 IDF 를 곱한다 .

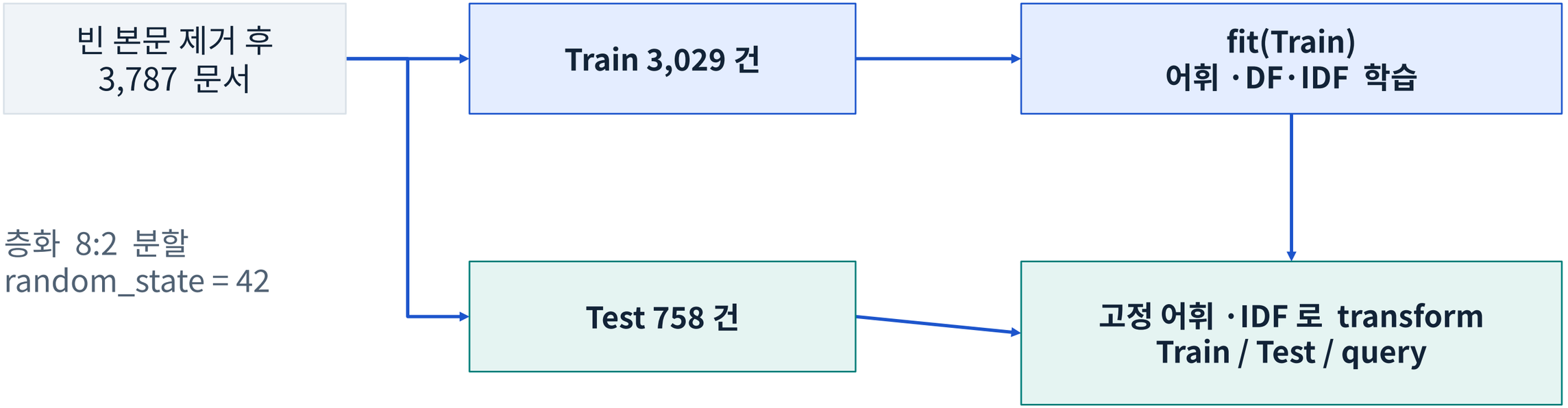
행별 계산

weighted 의 각 문서 행을 그 행의 L2 길이로 나눈다 . 0 벡터는 그대로 0 이다 .

fit 은 어휘 ·DF·IDF, count_matrix 는 TF, transform 은 가중치와 정규화를 담당한다

근거 : 평가 Q&A 및 저장소 실측 산출물 · 상세 출처는 마지막 장

2-2B. Train 에서 학습한 공간의 재사용



Test 를 fit 에 넣으면 어휘 선택과 IDF 에 평가 데이터의 분포가 섞인다 .

Test 에서 fit 을 다시 실행하면 정보 누출과 서로 다른 표현 공간 문제가 생긴다

근거 : 평가 Q&A 및 저장소 실측 산출물 · 상세 출처는 마지막 장

2-2C. 두 문서의 TF-IDF 계산

빈도 , IDF 곱셈 , 행별 정규화를 숫자로 따라간다

설명용 두 문서 : d_1 = “cat cat dog”, d_2 = “dog” 어휘 열 순서 : [cat, dog]

1. 빈도 TF

문서	cat	dog
d_1	2	1
d_2	0	1

× IDF
[1.405465, 1]

2. 원소별 곱

문서	cat	dog
d_1	2.810930	1
d_2	0	1

3. 행별 L2 정규화

문서	정규화 벡터
d_1	[0.942156, 0.335176]
d_2	[0, 1]

DF = [1, 2], N = 2 이므로 IDF(cat) = ln(3/2) + 1. d_1 의 L2 길이는 2.983509 이다 .

근거 : 평가 Q&A 및 저장소 실측 산출물 · 상세 출처는 마지막 장

2-3. 다른 데이터셋에 적용하는 설계

로더의 계약	(train_docs, y_train), (test_docs, y_test), target_names 를 반환한다 .
전처리 주입	CustomTfidfVectorizer(preprocessor=...) 로 tokenizer 와 불용어 정책을 교체한다 .
재학습 범위	데이터가 바뀌면 분할 · 어휘 ·IDF· 인덱스를 다시 만들고 단계별 검증을 반복한다 .
성능의 범위	한국어는 형태소 · 공백 토큰화 , OOV 와 클래스 분포를 점검한다 . 기존 정확도를 재사용할 수 없다 .

구성 요소의 재사용성과 학습된 인덱스의 자동 호환은 서로 다른 문제다

근거 : 평가 Q&A 및 저장소 실측 산출물 · 상세 출처는 마지막 장

3-1. TF·IDF 의 정의와 로그의 역할

TF(d,t) = 문서 d 의 단어 t 출현 횟수
DF(t) = 단어 t 를 포함하는 Train 문서 수
IDF(t) = ln[(1+N)/(1+DF(t))] + 1

TF-IDF 문서 안의 빈도 TF 에 코퍼스 전체의 희귀성 가중치 IDF 를 곱한다 .

로그와 smoothing 로그는 희귀성 비율의 큰 차이를 완화한다 . smoothing 은 안정된 계산 기준을 제공한다 .

숫자 예시 N=2 에서 DF=1 이면 IDF≈1.405465, DF=2 이면 IDF=1 이다 . 모든 문서에 나와도 0 이 아니다 .

이번 검증은 sklearn 의 Smooth IDF 정의와 정확히 맞춘다

근거 : 평가 Q&A 및 저장소 실측 산출물 · 상세 출처는 마지막 장

3-2A. 정규화와 코사인 · 거리의 관계

$$x_norm = x / \|x\|_2,\ q_norm = q / \|q\|_2$$
$$\cos(x,q) = x_norm \cdot q_norm$$
$$\|x_norm - q_norm\|_2^2 = 2 - 2\cos(x,q)$$

문서 길이 영향 정규화 전 내적 · 유클리드 거리는 벡터 크기의 영향을 받는다 . 코사인은 가중치의 방향을 비교한다 .

단위 벡터의 관계 L2 정규화한 비영벡터에서는 코사인 내림차순과 유클리드 거리 오름차순이 같은 순위를 준다 .

영벡터 처리 0 으로 나누지 않는다 . 빈 입력 ·OOV 는 0 벡터를 유지하고 검색 결과는 빈 리스트이다 .

코사인이 항상 우월한 척도라는 뜻이 아니라 현재 표현과 비교 목적에 맞춘 선택이다

근거 : 평가 Q&A 및 저장소 실측 산출물 · 상세 출처는 마지막 장

3-2B. 코사인 유사도의 숫자 예시

x=[3,4], q=[0,2] 인 두 비영벡터

계산 단계	계산 결과
원시 내적 $x \cdot q$	$3\times0+4\times2=8$
벡터의 길이	$\|x\|_2=5$, $\|q\|_2=2$
코사인	$8/(5\times2)=0.8$
정규화 벡터	x_norm=[0.6,0.8], q_norm=[0,1]
정규화 후 내적	$0.6\times0+0.8\times1=0.8$

문서 벡터를 한 번 정규화해 두면 질의 시 내적만 계산하면 된다

근거 : 평가 Q&A 및 저장소 실측 산출물 · 상세 출처는 마지막 장

3-3. TF-IDF 행렬 shape 의 의미

D_train
3,029 × 33,297

D_test
758 × 33,297

q
33,297

행은 문서 , 열은 Train 에서 확정한 단어 좌표이다 .

Test 와 query 도 동일한 열 순서를 사용한다 . OOV 단어는 새 열을 만들지 않는다 .
배치 쿼리 1 건은 (1, 33,297), 검색에 넘기는 단일 벡터는 (33,297,) 이다 .

검색 결과 벡터에는 Train 문서마다 점수 하나씩 , 총 3,029 개가 들어간다

근거 : 평가 Q&A 및 저장소 실측 산출물 · 상세 출처는 마지막 장

4-1. BoW 가 잃는 어순과 관계

```
from collections import Counter

Counter("dog bites man".split())
Counter("man bites dog".split())
#  두 문장의 단어별 빈도는 같다
```

구조적 한계

Unigram BoW 는 순서를 버린다 . 같은 빈도와 IDF 라면 의미가 달라도 같은 TF-IDF 가 된다 .

전처리와의 결합

not 을 제거하면 not good 과 good 의 차이까지 사라질 수 있다 .

실제 오류와 연결

ID 25 는 남은 주제 정보가 적었고 , ID 19 는 약어와 이전 대화 맥락을 충분히 반영하지 못했다 .

원인 해석은 실제 사례와 연결하고 Bigram · 문맥 모델의 개선 효과는 후속 실험으로 검증한다

근거 : 평가 Q&A 및 저장소 실측 산출물 · 상세 출처는 마지막 장

4-2. 사과의 여러 의미와 검색

과일

사과 과수원

주변 단어가 과일 의도를 나타낸다 .

기업

애플 신제품

개체명과 제품 문맥이 단서가 된다 .

용서

사과를 받아들이다

주변 구문이 다른 의미를 만든다 .

설명용 예시다 . 한국어 실측 , Apple· 사과 자동 치환 , 의미 구분은 현재 구현에 없다

근거 : 평가 Q&A 및 저장소 실측 산출물 · 상세 출처는 마지막 장

4-3. 백만 문서의 메모리 · 연산 병목

$$8 \times N \times |V| = 8 \times 10^6 \times 50{,}000$$
$$= 400 \times 10^9 \text{ bytes} \approx 372.5 \text{ GiB}$$

메모리 추정 float64 Dense 원소만 약 400GB 이다 . 객체 · 인덱스 등 부가 메모리는 포함하지 않았다 .

질의 연산 전체 내적은 $O(N|V|)$ 이다 . 희소 저장과 역색인으로 질의 단어 posting 과 후보를 줄인다 .

서비스 검증 구축 시간 · 메모리 · 지연 p50/p95 · 동시성 · 품질을 측정하고 캐시와 인덱스 갱신을 설계한다 .

현재 소규모 지연 측정은 백만 문서 부하 시험이나 p95 를 대신하지 못한다

근거 : 평가 Q&A 및 저장소 실측 산출물 · 상세 출처는 마지막 장

4-4A. 오류와 연결한 개선 실험

전처리부터 비교 부정어 보존 , 기술 용어 규칙 , Bigram 을 하나씩 바꾸고 실제 오류 유형의 변화를 확인한다 .

분류의 공정한 비교 같은 고정 분할에서 Accuracy · Macro-F1 · 클래스별 Precision · Recall 과 오류 사례를 대조한다 .

검증과 Test 분리 Train 내부 검증 분할로 설정을 고르고 최종 Test 는 마지막 확인에 사용한다 .

검색의 비교 조건 질의 · 관련도 등급 · 정답 작성 기준을 고정하고 TF-IDF · BM25 · 문맥 표현을 같은 조건에서 비교한다 .

현재 측정된 것은 분류 성능 · 순위 변화 · 알고리즘 동치성 · 시간이며 검색 관련도 품질은 후속 과제다

근거 : 평가 Q&A 및 저장소 실측 산출물 · 상세 출처는 마지막 장

4-4B. 검색 품질의 정량 평가

Precision@5 = Top-5 중 관련 문서 수 / 5
DCG@K = Σ(i=1..K) (2^rel_i − 1) / $\log_2(i+1)$
nDCG@K = DCG@K / IDCG@K

Precision 과 Recall	Precision@5 는 상위 결과의 관련 비율이다 . Recall@K 는 정답 문서 중 얼마나 회수했는지 본다 .
nDCG	관련도 등급과 순위를 함께 평가한다 . 높은 관련도 문서를 위에 놓을수록 DCG 가 커진다 .
현재 측정 범위	Precision@5 · nDCG@5 는 아직 미측정이다 . Raw 와의 겹침률이나 전체 / 역색인 순위 일치율로 대체할 수 없다 .

관련도 정답셋이 있어야 검색 품질 수치를 주장할 수 있다

근거 : 평가 Q&A 및 저장소 실측 산출물 · 상세 출처는 마지막 장

5-1. TF 변형의 수식과 비교 실측

동일 분할 · 어휘 ·IDF·L2 정규화에서 비교한 결과

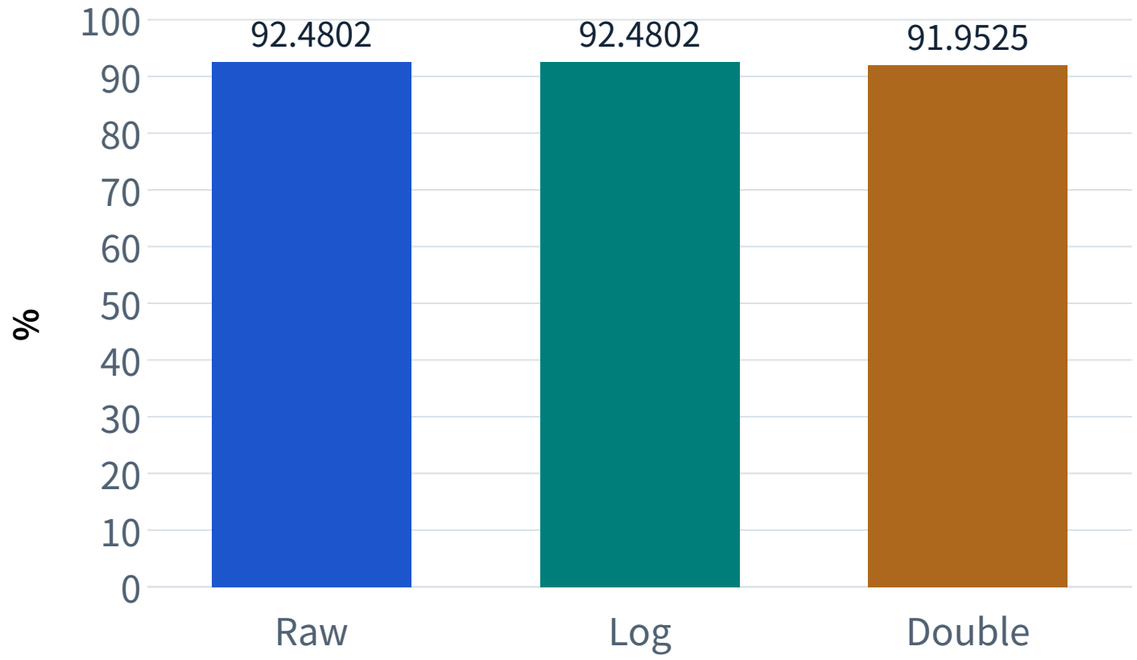
TF 방식	변환식	Accuracy	Macro-F1	Raw 겹침
Raw	c	92.4802%	0.925329	100%
Log	ln(1+c)	92.4802%	0.925139	60%
Double	0.5+0.5c/cmax (c>0)	91.9525%	0.919962	40%

Double 은 c=0 과 빈 문서를 0 으로 유지한다 . 겹침은 우주 질의 Top-5 구성의 비교값이다

근거 : 평가 Q&A 및 저장소 실측 산출물 · 상세 출처는 마지막 장

5-1B. TF 변형의 분류와 검색 결과

동일 데이터 · 분할 · 분류기로 비교한 원문 실측



TF	Macro-F1	Top-5 겹침
Raw	0.925329	100%
Log	0.925139	60%
Double	0.919962	40%

동일 분할 · 분류기로 비교한 원문 실측이다 . Top-5 겹침은 우주 질의의 Raw 결과 대비다 .
순위 구성이 달라졌다는 사실만으로 검색 품질 향상을 판단할 수 없다 .

근거 : 평가 Q&A 및 저장소 실측 산출물 · 상세 출처는 마지막 장

5-2A. BM25 의 TF 포화와 길이 보정

$$IDF(t) = \ln[1 + (N - DF(t) + 0.5)/(DF(t) + 0.5)]$$
$$K(D) = k_1[1 - b + b|D|/avgdl]$$
$$s(D,Q) = \Sigma_t\, q_t \cdot IDF(t) \cdot f_t,D(k_1+1) \,/\, [f_t,D + K(D)]$$

매개변수 k_1=1.5, b=0.75 를 사용한다 . TF 증가의 기여를 포화시키고 평균 길이 대비 문서 길이를 보정한다 .

독립 수식 검증 세 문서 Toy 기대 점수는 [1.5908508973, 0.4700036292, 0]. 최대 오차는 2.2204e−16 이다 .

비교 범위 실제 네 질의에서 TF-IDF 와 BM25 의 Top-5 문서 · 점수를 비교했다 . 점수의 절댓값 척도는 서로 다르다 .

수식 구현의 정확성과 관련도 기반 검색 품질의 개선은 각각 검증해야 한다

근거 : 평가 Q&A 및 저장소 실측 산출물 · 상세 출처는 마지막 장

5-2B. 반복 질의어의 구현 차이

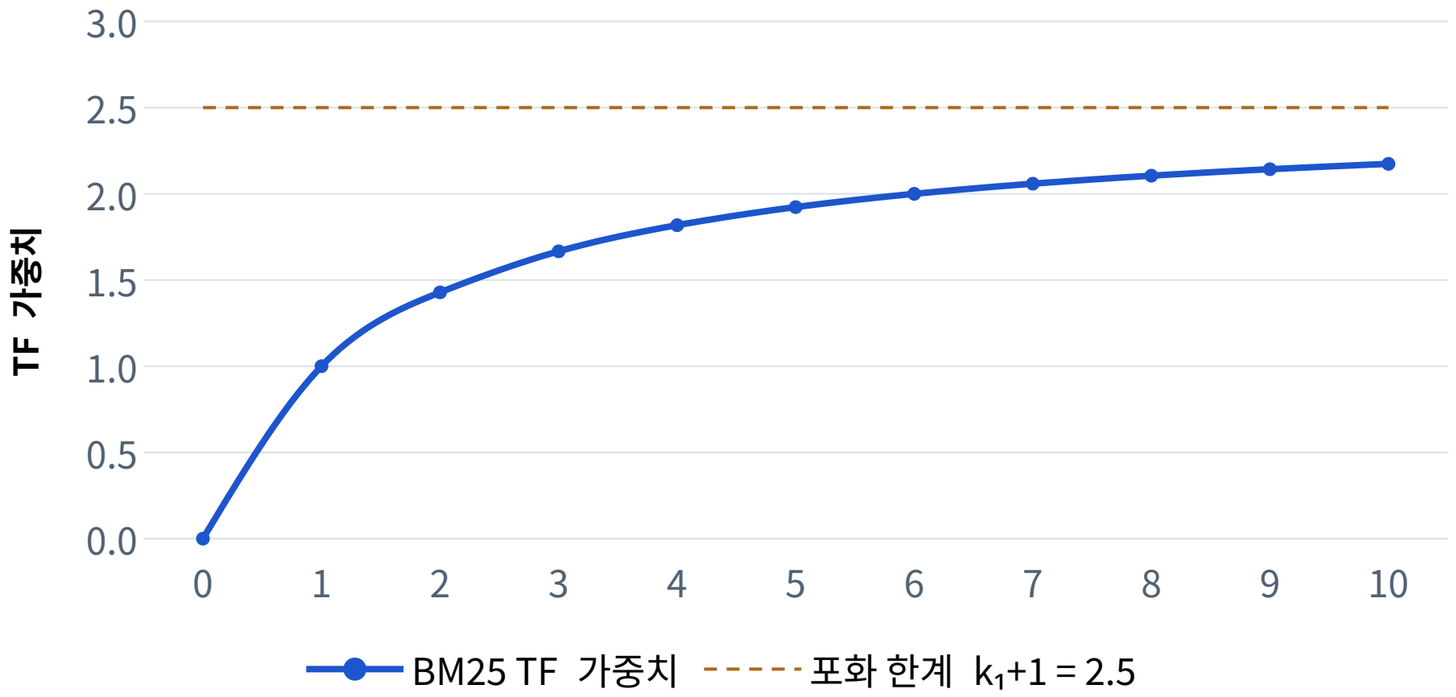
실행 경로	반복어 처리	해석
CLI BM25Okapi.score	질의 빈도 q_t 를 곱함	같은 단어를 반복하면 기여가 증가
노트북 BonusBM25	고유 질의 단어를 한 번만 더함	같은 단어 반복을 한 번으로 취급
현재 비교한 네 질의	반복어가 없음	이 정책 차이가 해당 결과에는 영향 없음

CLI 와 노트북이 모든 입력에서 완전히 같은 BM25 규칙이라고 설명하면 안 된다

근거 : 평가 Q&A 및 저장소 실측 산출물 · 상세 출처는 마지막 장

5-2C. BM25 의 TF 포화 곡선

단어가 반복될수록 증가폭이 줄어든다



$g(f) = 2.5f / (f + 1.5)$

$k_1 = 1.5$, $|d| = avgdl$ 가정

f = 1 이면 1.0
f = 3 이면 1.667
f 가 커지면 2.5 에 접근

반복 횟수의 추가 기여가
점점 작아진다 .

수식을 시각화한 설명용 계산이다 . 실험 성능이나 검색 관련도 측정값이 아니다 . 가로축은 문서 내 빈도 f 다 .

근거 : 평가 Q&A 및 저장소 실측 산출물 · 상세 출처는 마지막 장

5-3A. 역색인의 점수 계산과 복잡도

$score_j = \Sigma_{t:q_t \neq 0} X_{j,t} q_t$
Dense: O(N|V|)
현재 역색인 : O(N + P_Q) + Top-K 비용

posting 구성 각 단어마다 그 단어가 등장하는 문서 ID 와 정규화된 TF-IDF 가중치를 저장한다 .

질의 처리 질의의 비영 단어에 해당하는 posting 만 순회하며 문서별 점수에 $X_{j,t} q_t$ 를 누적한다 .

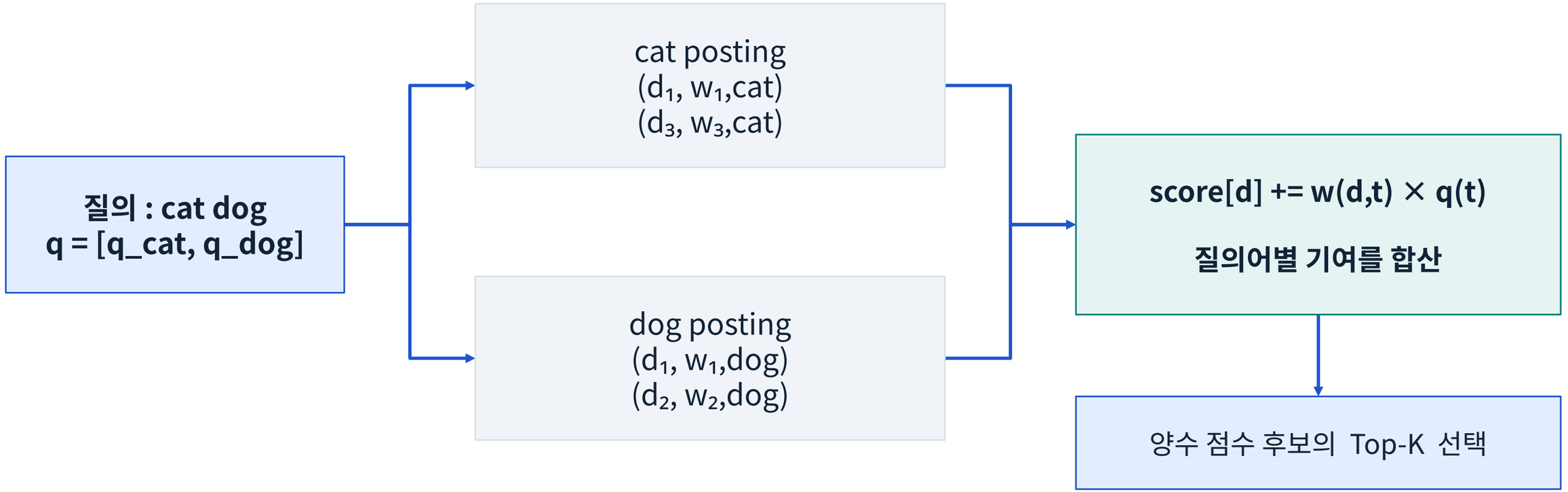
구현상 남는 비용 전체 점수 배열 초기화에 O(N) 이 든다 . Top-K 후보 선택과 경계 동점 정렬 비용도 추가된다 .

7 질의에서 전체 스캔과 점수 최대 오차 0, Top-5 순서 100% 일치를 확인했다

근거 : 평가 Q&A 및 저장소 실측 산출물 · 상세 출처는 마지막 장

5-3B. 역색인의 posting 누적

질의어가 나타난 문서의 가중치만 합산한다

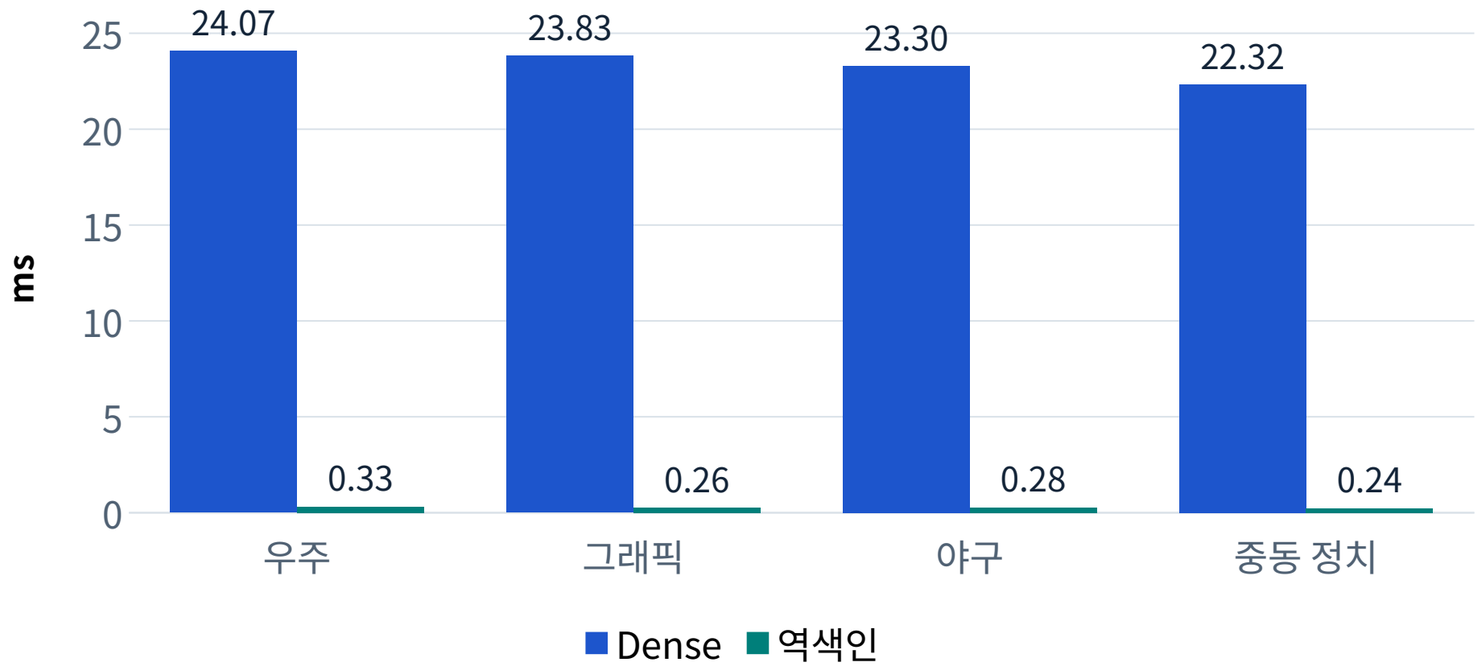


P_Q = Σ df(t). 현재 코드는 길이 N 의 점수 배열을 매 질의 초기화하므로 O(N + P_Q) 와 순위 선택 비용을 가진다 .

근거 : 평가 Q&A 및 저장소 실측 산출물 · 상세 출처는 마지막 장

5-3C. 역색인의 시간과 측정 조건

우주 질의 , 점수 계산과 Top-5 선택의 원문 실측



73.3 배

우주 질의의 원문 측정
24.0746 ms / 0.3283 ms

7 개 검증 질의
점수 최대 오차 0
Top-5 순위 모두 일치

점수 계산 · Top-5 선택 포함 , 토큰화 · 벡터화 제외 . BLAS 2 스레드 , 3 회 실행 중앙값 .
인덱스 구축 0.463321 초는 질의 시간과 별도다 . 하드웨어 · 쿼리에 따라 달라진다 .

근거 : 평가 Q&A 및 저장소 실측 산출물 · 상세 출처는 마지막 장

06. 구술 시연 순서와 확인 명령

```
python verify_tfidf.py
python main.py --build-index
python main.py --query "space satellite launch orbit" --topk 5
python main.py --train-classifier
python main.py --bonus
```

명령은 프로젝트 루트에서 실행하며 같은 가상환경의 Python 을 사용한다

구현 근거

작은 TF·DF·IDF 계산과 참조 오차를 설명한 뒤 검색 결과 · 빈 입력 · 재로딩을 보여 준다 .

결과와 한계

Accuracy·Macro-F1· 혼동 행렬을 실제 다섯 오류와 연결하고 남은 검색 품질 평가를 구분한다 .

보너스

TF 변형 , BM25, 역색인의 수식 · 실측 · 비교 조건을 설명한다 . 전체 보너스는 분류 세 번을 포함한다 .

근거 : 평가 Q&A 및 저장소 실측 산출물 · 상세 출처는 마지막 장

6-1. 출처와 수치 해석 기준

문서와 실제 구현 , 공개 실측 산출물을 함께 사용했다

문서	docs/A4_1_eval_QnA.md 브랜치 : docs/a4-1-final-readme-qna
구현	src/preprocess.py · src/tfidf.py · src/search_engine.py src/classifier.py · src/bonus/ · A4_1_mission.ipynb
수치와 그림	artifacts/dataset_metadata.json · tfidf_verification.json classifier_metrics.json · bonus_metrics.json · confusion_matrix.png
해석 범위	실측은 저장소의 2026-11-12 기록이다 . 계산 예제는 설명용이다 . 개선 효과와 검색 관련도 지표는 동일 조건의 추가 실험이 필요하다 .

https://github.com/ntt65/A4_1/tree/docs/a4-1-final-readme-qna

근거 : 평가 Q&A 및 저장소 실측 산출물 · 상세 출처는 마지막 장